PROJECT PROPOSAL: INTELLIGENT NAVIGATION AND REAL-TIME OBSTACLE AVOIDANCE IN AUTONOMOUS ROBOTS USING DEEP REINFORCEMENT LEARNING

Domain: Robotics & Automation (Task 4)

Topic: Role of AI in Autonomous Robots

Format: Mini Project Proposal

Target Length: 5–6 Pages (~2,500 words)

1. Project Abstract

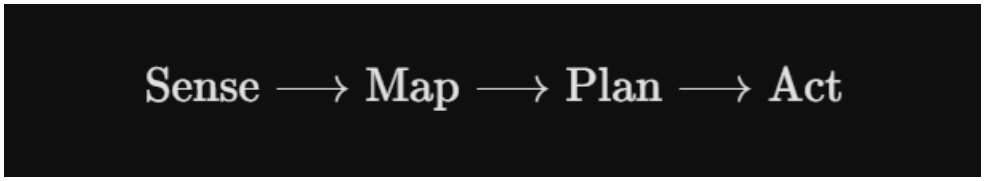
Traditional autonomous mobile robots (AMRs) rely on rigid, hard-coded rules and classical path-planning algorithms like A* or Dijkstra's. While effective in static environments, these methods fail in complex, highly dynamic, and unstructured real-world settings. This proposal outlines the development of an Artificial Intelligence (AI) framework utilizing Deep Reinforcement Learning (DRL) paired with Deep Q-Networks (DQN) and Proximal Policy Optimization (PPO) to govern robot locomotion.

By bypassing traditional mapping phases and processing raw sensor streams directly into motor commands, the proposed system enables a differential-drive robot to navigate unknown environments autonomously. The project details the system architecture, mathematical formulations, simulation pipeline, evaluation metrics, and project management timeline required for full-scale development.

2. Introduction and Background

Autonomous robots have evolved from stationary factory line tools into mobile agents working in warehousing, domestic care, defense, and planetary exploration. The critical bottleneck facing modern robotics is **perception-to-action latency** in shifting environments.

Classical robotics relies on a sequential paradigm:



If a dynamic obstacle appears suddenly, the robot must rebuild its cost-map and recalculate paths, which degrades performance and increases collision risks.

Integrating Artificial Intelligence—specifically deep learning and reinforcement learning—allows for an **End-to-End** operational structure:

$$\text{Sense (Raw Sensor Input)} \longrightarrow \text{Deep Neural Network (AI)} \longrightarrow \text{Act (Motor Control Volts)}$$

This approach enables the robot to learn optimal behaviors through iterative trial-and-error within a simulated workspace, optimizing paths dynamically much like biological organisms.

3. Problem Statement

Current obstacle avoidance mechanisms struggle with the velocity profiles of moving entities (e.g., humans walking in a factory or warehouse). Classical edge-detection systems frequently suffer from computational bottlenecks when interpreting high-density 3D cloud points under tight real-time deadlines.

Consequently, there is a distinct need for an AI-driven control system capable of mapping low-latency sensor inputs (2D LiDAR ranges and IMU telemetry) directly onto continuous kinematic outputs. This must be accomplished while minimizing energy consumption and ensuring zero collisions.

4. Project Objectives

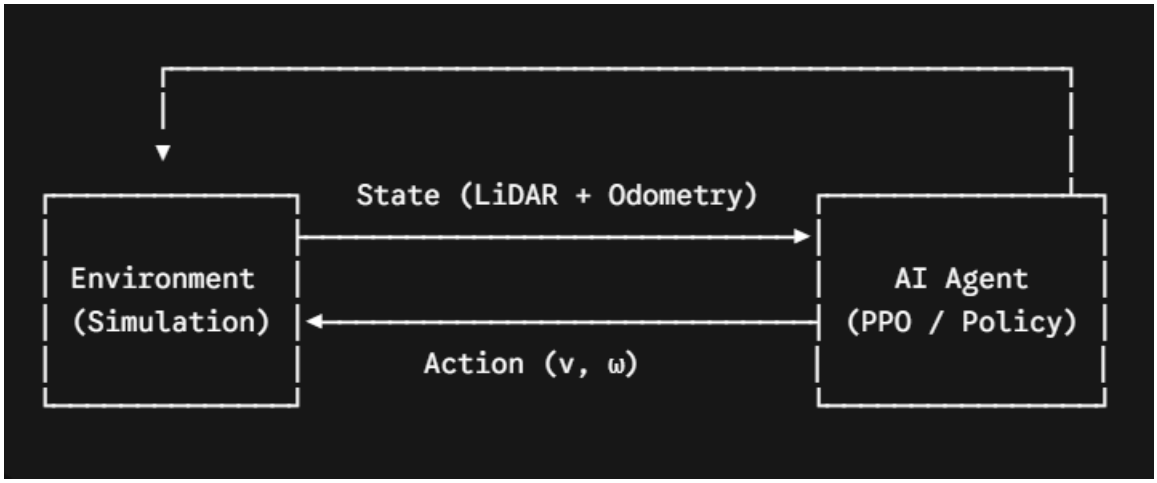
The core milestones of this project proposal include:

1. **Developing a High-Fidelity Simulation Workspace:** Create an unstructured dynamic environment using ROS 2 (Robot Operating System) and Gazebo.
2. **Designing a Multi-Modal AI Brain:** Architect a Deep Neural Network that fuses spatial rangefinder information with positional telemetry data.
3. **Implementing a DRL Control Policy:** Implement and optimize a Proximal Policy Optimization (PPO) agent to manage local path planning.

4. **Physical Deployment Testing (Sim-to-Real):** Evaluate the sim-to-real transferability of the policy using a physical TurtleBot3 platform to assess its real-world viability.

5. Proposed Methodology and Architecture

The system utilizes a closed-loop feedback loop where the environment rewards or penalizes the AI agent based on safety metrics and target proximity.



5.1 Robot Kinematics and Simulation Platform

The target platform is a differential-drive mobile robot. Its forward kinematics are modeled by:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}$$

Where:

- x, y are the planar coordinates.
- θ is the heading angle.
- v is linear velocity.
- ω is angular velocity.

5.2 Deep Reinforcement Learning Framework

The AI model treats navigation as a Markov Decision Process (MDP) defined by the tuple (S, A, P, R, γ):

State Space (S): Consists of a downsampled 26-beam 2D LiDAR range vector $(d_1 \text{ to } d_{26})$, current linear/angular velocities, the distance remaining to the goal (D_g), and the relative heading angle to the target (ϕ_g)

- **Action Space (A):** Continuous control outputs controlling linear velocity $v \in [0.0, 0.26 \text{ m/s}]$ and angular velocity $\omega \in [-1.0, 1.0 \text{ rad/s}]$.
- **Reward Function (R):** The critical component guiding policy convergence. It is defined mathematically as:

$$R_t = R_{\text{goal}} + R_{\text{collision}} + \alpha \cdot (D_{g,t-1} - D_{g,t}) - \beta \cdot \|\omega_t\|$$

Where:

- $R_{\text{goal}} = +1000$ if the target is successfully reached.
- $R_{\text{collision}} = -500$ if sensor metrics register a crash.
- $\alpha \cdot (D_{g,t-1} - D_{g,t})$ provides a positive reward for moving toward the goal.
- $-\beta \cdot \|\omega_t\|$ penalizes rapid, erratic rotation to ensure smooth trajectories.

6. Technical Specifications and Hardware/Software Stack

6.1 Software Infrastructure

- **Operating System:** Ubuntu 22.04 LTS (Jammy Jellyfish).
- **Middleware:** ROS 2 (Humble Hawksbill) for sensor node management and motor actuation execution.
- **Simulation Engine:** Gazebo 11 for physical dynamics and sensor replication.

- **AI Engine:** PyTorch paired with Stable-Baselines3 for DRL training operations.

6.2 Hardware Prototype Layout

For physical validation, the proposal specifies:

- **Microprocessing Core:** NVIDIA Jetson Orin Nano (8GB) to execute model inference locally at the edge.
- **Primary Sensors:** RPLIDAR A1 (360-degree laser scanner) and an on-board MPU6050 IMU.
- **Actuation System:** Dual Dynamixel XL430 smart servo motors.

7. Implementation Plan and Milestones

The project will be executed across five distinct phases over a 16-week timeline:

Phase 1: Environment Generation & Setup (Weeks 1–3)

- Install ROS 2, Gazebo, and required deep learning packages.
- Design virtual test environments featuring both static barriers and dynamic obstacles.

Phase 2: Agent Design & Reward Structuring (Weeks 4–7)

- Construct the multi-layer neural network architecture in PyTorch.
- Establish data transmission pipelines between ROS 2 sensor topics and the PyTorch model input layers.

Phase 3: Policy Training & Simulation (Weeks 8–11)

- Run training routines over 2 million timesteps within the Gazebo environment.
- Fine-tune hyperparameters (learning rates, clip ranges, discount factors γ) to prevent policy divergence.

Phase 4: Hardware Integration & Deployment (Weeks 12–14)

- Quantize the trained PyTorch network via TensorRT to run efficiently on edge hardware.
- Perform physical field tests with the mobile robot prototype.

Phase 5: Evaluation & Report Formulation (Weeks 15–16)

- Compile performance metrics, generate comparative plots, and draft the final project report.

8. Expected Outcomes and Evaluation Metrics

The project anticipates an edge-computed AI policy capable of safe navigation in environments it has never encountered before. Success will be measured using three key metrics:

1. **Success Rate:** The percentage of test runs where the robot successfully reaches its destination without collisions. The target metric is ≥95%
2. **Path Efficiency Factor (Ep):** A ratio comparing the actual distance traveled (D actual) against the ideal straight-line distance (D ideal):

$$E_p = \frac{D_{\text{ideal}}}{D_{\text{actual}}}$$

3. **Average Processing Latency:** The execution time required for the neural network to calculate motor outputs from sensor inputs. The target performance benchmark is ≤20ms per step.

9. Potential Challenges and Risk Mitigation

- **The Sim-to-Real Gap:** Policies trained in perfect simulations often struggle with real-world sensor noise and friction.
    - *Mitigation:* Apply domain randomization during training, adding artificial noise to simulated laser scans and varying structural physics properties (like surface friction).
- **High Computational Demand:** Training deep reinforcement models locally can be slow and resource-intensive.
    - *Mitigation:* Offload initial training iterations to cloud-hosted GPU instances (e.g., Google Colab Pro or AWS EC2), saving local edge deployment exclusively for model inference testing.

10. Conclusion

This proposal presents an actionable methodology for embedding AI directly into the locomotion frameworks of autonomous mobile machinery. Shifting

away from rigid rule-based code toward a self-learning Deep Reinforcement Learning paradigm makes autonomous systems substantially more adaptive, resilient, and safer in human workspaces. Successful execution of this project will contribute practical insights into edge-based AI deployments for modern logistics and service robotics applications.

11. References

- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.
- Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66).